Data Engineer_Бондарев Иван Романович

Ответы на задания

Задача 1. Преобразовал бы таблицу в следующий вид:

```
CREATE TABLE clients (
    client_id SERIAL primary key,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(70) NOT NULL,
    birth_date DATE NOT NULL,
    gender CHAR(1) CHECK (gender in ('M','F')),
    login VARCHAR(50) UNIQUE NOT NULL,
    city_id INTEGER NOT NULL REFERENCES city(city_id),
    registration_date TIMESTAMP default CURRENT_TIMESTAMP NOT NULL,
    is_active BOOLEAN NOT NULL default TRUE
    );
```

Добавил первичный ключ `client_id`, чтобы обеспечить однозначную идентификацию строки в таблице и целостность данных.

С целью соблюдения бизнес ограничений уменьшил длину хранимых символов в имени `first_name` и фамилии `last_name` до 50 и 70.

Убрал столбец age, так как не логично его содержать в таблице, поскольку с течением времени он становится неактуальным. Лучше иметь поле Дата рождения `birth_date` и уже от него, при необходимости, вычислять возраст клиента.

Добавил поле `gender`, что облегчит сбор данных, сегментов, аналитики.

Поле `city_id` сделал внешним ключом для предполагаемой таблицы City на ее первичный ключ.

Увеличил длину `login` до 50 символов и ограничение по уникальности.

Дату регистрации на сайте изменил имя reg, как не однозначное на `registration_date`.

Также чтобы не удалять клиента, отслеживания статуса, аналитики и возможности формирования сегментов добавил признаки Активен/Не активен `is_active`.

Задача 2. Здесь предложил бы разделить данные о товаре и цене на 2 таблицы, для того, чтобы иметь возможность сохранять историю цены

Основная таблица товаров без цены:

```
CREATE TABLE items (
    item_id SERIAL primary key,
    item_name VARCHAR(255) NOT NULL
);
```

Здесь будет редко изменяемы данные о товаре, цену удаляю, чтобы избежать дублирования.

Новая таблица с ценой для хранения истории цены:

```
CREATE TABLE item_prices (
    price_id SERIAL primary key,
    item_id INTEGER NOT NULL REFERENCES items(item_id),
    price NUMERIC(10,2) NOT NULL CHECK (price>0.00),
    date_from TIMESTAMP NOT NULL,
    date_to TIMESTAMP NOT NULL DEFAULT '3000-12-31 23:59:59',
    is_current BOOLEAN NOT NULL default TRUE,
    CONSTRAINT check_dt CHECK (date_from<date_to),
    );

CREATE UNIQUE INDEX uniq_current_price on item_prices (item_id)
where is_current = TRUE;
```

Здесь создал уникальный первичный ключ цены `price_id`, указал внешний ключ на товар `item_id`, цену с плавающей точкой `price`. Самое важное здесь поля `date_from` и `date_to`, которые показывают период действия цены. Текущая цена будет иметь `date_to` равный 3000 году. Это позволяет сохранять историю, когда цена изменится: `date_to` с `'3000-12-31:23:59:59'` изменим на текущую дату `CURRENT_TIMESTAMP` (и эта запись станет старой ценой). Также `is_current` нужно поменять на FALSE, а новая строка с новой ценой будет иметь `date_from` текущей датой установления новой цены, а `date_to` станет равным `'3000-12-31:23:59:59'`. Затем добавил проверку на корректность дат, чтобы дата начала всегда была меньше даты окончания действия цены. Еще добавил индекс для защиты данных, чтобы не возможно было вставить 2 одинаковые текущие цены на 1 товар `item_id`.

Такое разделение таблиц будет похоже на принцип SCD 2, при котором мы не теряем историю цены. Если изменений цены не много, можно делать все в одной таблице и дублировать записи, но для нормализации в 3NF лучше иметь отдельные таблицы.

Задача 3. Я бы переписал этот код так:

```
SELECT d.department_id,
       d.department_name,
       SUM(e.salary) as total_salary
FROM departments d
INNER JOIN employee e
       ON d.department_id=e.department_id
INNER JOIN locations l
       ON d.location_id=l.location_id
WHERE l.city='Seoul' AND e.manager_id IS NULL
GROUP BY d.department_id, d.department_name
HAVING SUM(e.salary)>100000;
```

Я заметил следующие ошибки в предложенном коде:

- отсутствие части элиасов, добавил чтобы было однозначно понятно к каким таблицам привязаны столбцы.

- старый стиль написания соединения таблиц, написал через INNER JOIN.

- заменил подзапрос с location на JOIN для более быстрого выполнения запроса

- ошибка с NULL. Не MANAGER_ID=NULL, а е. MANAGER_ID IS NULL.

- использование агрегатной функции в WHERE, правильно использовать SUM в группировке с HAVING

- группировка проведена не по всем столбцам, все не агрегированные поля должны быть в GROUP BY.

- не правильное написание таблицы `employees`.

- я написал условие строго больше 100000, как в условии задачи, а не в коде.

Задача 4. Здесь я предложил бы 3 варианта:

Вариант 1. Предполагаю, что данных не хватает. Тогда надо уточнить данные. Нужно понять: где лежит карандаш и точно ли нужно перепрыгивать? Карандаш маленький, человек большой. Прыгать через карандаш – это чрезмерное усилие, избыточное. Достаточно просто перешагнуть. Можно ли карандаш убрать или обойти?

Вариант 2. Если я принимаю условия как данность и данных достаточно. Тогда можно предположить, что карандаш либо закатился под большое препятствие, например, шкаф и через него крайне трудно, невозможно перепрыгнуть. Либо он укатился к стене, тогда тоже нельзя перепрыгнуть.

Вариант 3. Возможно, человек по внутренним причинам не может перепрыгнуть карандаш, глубоко личные убеждения, не позволяющие ему прыгать через карандаши. ☺